Class06

Ami Pant (A15991358)

Table of contents

All functions in R have at least 3 things:

- A **name**, we pick this and use it to call our function
- Input **arguments** (there can be multiple)
- The **body**lines of R code that do the work

Our first (silly) function

Write a function to add some numbers

```
add <- function(x, y = 1) {
  x + y
}
```

Now we can call this function:

```
add(c(10, 10), 100)
```

```
[1] 110 110
```

```
add(10, 100)
```

```
[1] 110
```

A second function

Write a function to generate random nucleotide sequences of a user specifies length:

The `sample()` function can be helpful here.

```
v <- sample(c("A", "C", "G", "T"), size = 50, replace = T)
```

I want a 1 element long character vector that looks like this "CACAGC" not "C" "A" "C" "G" "C"

```
paste(v, collapse = "")
```

```
[1] "AGTTTATTTTGGTCTCTCCCCACCACTTTGTTCATTACAAAATAGAATGT"
```

Turn this into my first wee function

```
generate_dna <- function(size = 50) {
  v <- sample(c("A", "C", "G", "T"), size= size, replace = T)
  paste(v, collapse = "")

}
```

Test it:

```
generate_dna(60)
```

```
[1] "CGTCGTTGGGATGACAATCTGGCTATGTTTTCATTCCGAATCGATCGACTTGCTCCTCGT"
```

```
fasta <- F
if(fasta) {
  cat("HELLO You!")
} else {
  cat("NO you dont!")
}
```

```
NO you dont!
```

Add the ability to return a multi-element vector or a single element fasta like vector

```
generate_fasta <- function(size = 50, fasta = T) {

v <- sample(c("A", "C", "G", "T"), size= size, replace = T)
s <- paste(v, collapse = "")

  if (fasta) {
    return(s)
  } else {
    return(v)
  }
}
```

A protein generating function

```
generate_protein <- function(size = 50, fasta = T) {
 aa  <- sample(c("A","R","N","D","C","Q","E","G","H","I",
          "L","K","M","F","P","S","T","W","Y","V"), size = size, replace = T)
 s <- paste(aa, collapse = "")

  if (fasta) {
    return(s)
  } else {
    return(aa)
  }

}
```

```
generate_protein(6)
```

```
[1] "GIQDCH"
```

Use our new `generate_protein()` function to make random protein sequences of length 6 to 12 (i.e. one length 6, one length 7, etc. up to length 12)

```
generate_protein(6)
```

```
[1] "EMTLSH"
```

```
generate_protein(7)
```

```
[1] "QQPWPAD"
```

```
generate_protein(8)
```

```
[1] "PRPTNEAN"
```

```
generate_protein(9)
```

```
[1] "CKTRHMQTC"
```

```
generate_protein(10)
```

```
[1] "TALFYVHDAW"
```

```
generate_protein(11)
```

```
[1] "WPNCECRIMAE"
```

```
generate_protein(12)
```

```
[1] "CCHYYKYETIIL"
```

A second way is to use a `for()` loop:

```
lengths <- 6:12
lengths
```

```
[1]  6  7  8  9 10 11 12
```

```
for (i in lengths) {
  cat(">", i, "\n", sep = "")
  aa <- generate_protein(i)
  cat(aa)
  cat("\n")
}
```

```
>6
LTEWYV
>7
GAGSRVC
>8
QPMTEDDT
>9
GGRKFQVPN
>10
VIPPAKHCWN
>11
SGFWAKTCPLT
>12
QWSESLTGPVEE
```

A third, and better way to solve this is to use the `apply()` family of functions, specifically the `sapply()` function in this case.

```
sapply(6:12, generate_protein)
```

```
[1] "VEHCPL"       "SWGHCDI"      "QHYFPGNL"     "PPSDDDACF"    "MKYMKEVGFY"
[6] "NDSASYYFIPA"  "FGFCAAKYWLQW"
```